

TIPT-ViTPose: specifica sintetica per prototipo Hugging Face

Base: `transformers.VitPoseForPoseEstimation` + COCO2017 keypoints

1. Obiettivo

Realizzare un prototipo **Texture-Invariant Pose Transformer** basato su **ViTPose** usando Hugging Face come base. Il modello deve riutilizzare pesi e head di ViTPose quando possibile, aggiungendo solo:

- uno **shape stream leggero** costruito da una vista strutturale dell'immagine;
- un modulo di **shape-guided cross-attention**;
- una loss custom per training su COCO2017, dato che il forward Hugging Face di ViTPose non implementa direttamente il training con labels.

La prima versione deve essere semplice, stabile e confrontabile con la baseline ViTPose HF.

2. Base Hugging Face da usare

Usare come punto di partenza:

```
from transformers import AutoImageProcessor, ViTPoseForPoseEstimation
processor = AutoImageProcessor.from_pretrained("usyd-community/vitpose-base-simple")
base_model = ViTPoseForPoseEstimation.from_pretrained("usyd-community/vitpose-base-simple")
```

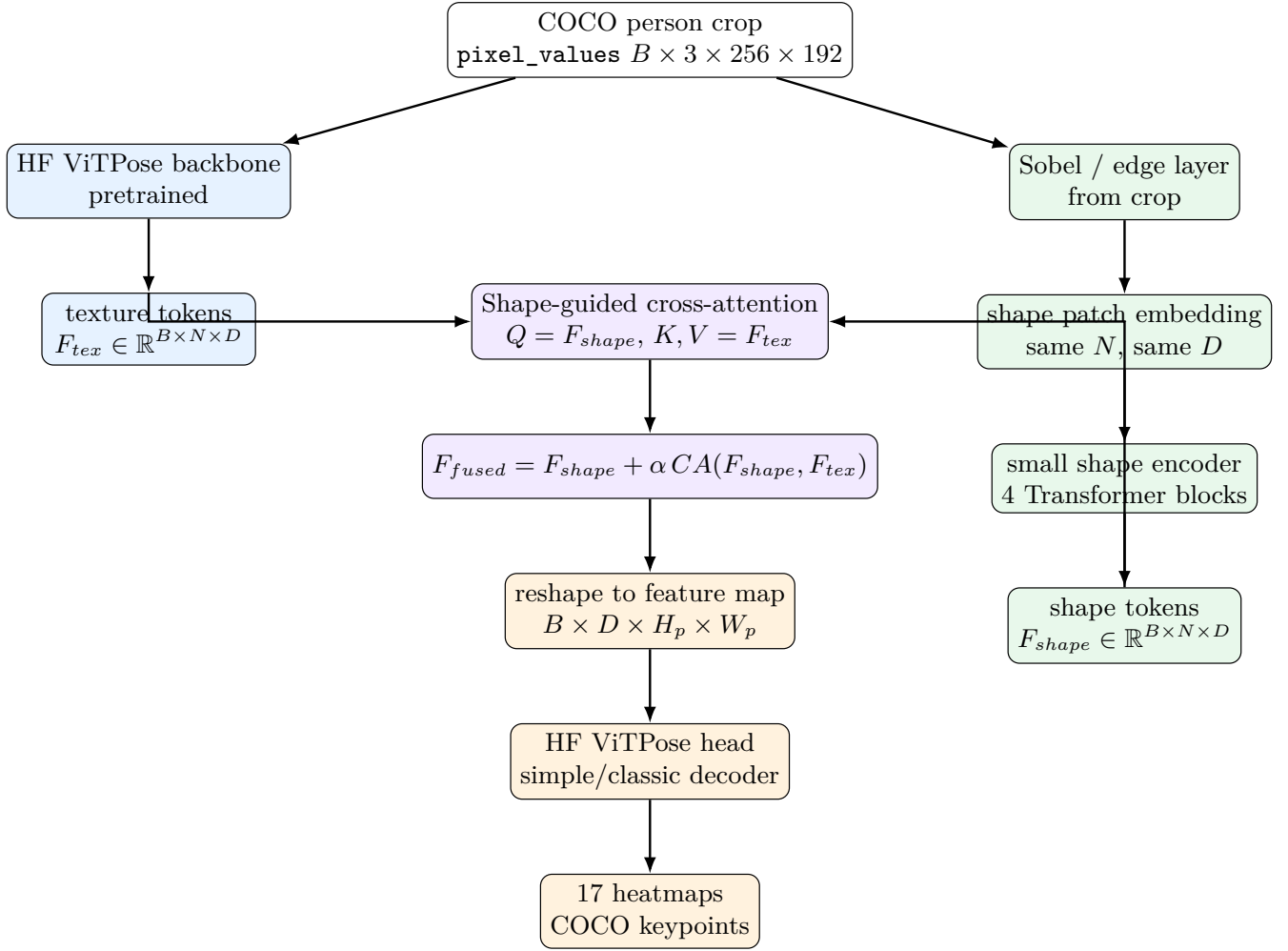
La classe HF espone:

- `base_model.backbone`: backbone ViT usato come **texture stream**;
- `base_model.head`: decoder/head ViTPose da riutilizzare per produrre heatmap;
- output heatmaps;
- feature maps finali in `outputs.feature_maps[-1]` dentro il forward ufficiale.

Nel codice HF, il training con labels solleva `NotImplementedError`; quindi implementare una loss esterna nel training loop.

3. Architettura consigliata: late fusion minimale

Questa è la versione da implementare per prima.



4. Moduli da implementare

4.1 SobelEdgeLayer

Input: `pixel_values` normalizzati, shape $B \times 3 \times H \times W$. Output consigliato: $B \times 1 \times H \times W$.

- Convertire a grayscale con media pesata o media semplice sui canali.
- Applicare Sobel x e y con conv fissa non trainabile.
- Restituire edge magnitude normalizzata: $E = \sqrt{E_x^2 + E_y^2} + \epsilon$.

4.2 ShapePatchEmbedding

Usare una `Conv2d` con `kernel_size=patch_size`, `stride=patch_size`, output channels D .

- Deve produrre lo stesso numero di token del backbone ViTPose.
- Aggiungere positional embedding imparabile o riusare forma compatibile.
- Tenere $D_{shape} = D_{tex}$ nella prima versione.

4.3 ShapeEncoder

Encoder piccolo: 4 Transformer blocks standard PyTorch.

- dimensione: D uguale al backbone;
- num heads: uguale o divisore compatibile con D ;
- MLP ratio: 4;
- dropout basso o nullo nella prima prova.

4.4 ShapeGuidedCrossAttention

Modulo centrale:

```
attn_out, _ = self.cross_attn(
    query=F_shape, # [B, N, D]
    key=F_tex,     # [B, N, D]
    value=F_tex,   # [B, N, D]
    need_weights=False,
)
F_fused = F_shape + self.alpha * attn_out
```

Usare `nn.MultiheadAttention(embed_dim=D, num_heads=h, batch_first=True)`. Inizializzare `alpha` a 0.1 per non disturbare troppo la head ViTPose all'inizio.

5. Classe modello consigliata

Creare una classe wrapper, senza modificare direttamente il package `transformers`.

```
class TiptVitPoseForPoseEstimation(nn.Module):
    def __init__(self, checkpoint="usyd-community/vitpose-base-simple"):
        super().__init__()
        self.vitpose = VitPoseForPoseEstimation.from_pretrained(checkpoint)
        self.backbone = self.vitpose.backbone # texture stream
        self.head = self.vitpose.head # reuse decoder
        self.edge = SobelEdgeLayer()
        self.shape_embed = ShapePatchEmbedding(...)
        self.shape_encoder = ShapeEncoder(depth=4, ...)
        self.cross_fusion = ShapeGuidedCrossAttention(...)

    def forward(self, pixel_values, dataset_index=None, flip_pairs=None):
        tex_outputs = self.backbone.forward_with_filtered_kwargs(
            pixel_values, dataset_index=dataset_index
        )
        F_tex = tex_outputs.feature_maps[-1] # [B, N, D]

        edge = self.edge(pixel_values) # [B, 1, H, W]
        F_shape = self.shape_embed(edge) # [B, N, D]
        F_shape = self.shape_encoder(F_shape) # [B, N, D]

        F_fused = self.cross_fusion(F_shape, F_tex) # [B, N, D]
        fmap = tokens_to_map(F_fused, config) # [B, D, Hp, Wp]
        heatmaps = self.head(fmap, flip_pairs=flip_pairs)
        return {"heatmaps": heatmaps, "F_shape": F_shape, "F_tex": F_tex}
```

6. COCO2017 training/test

Usare approccio top-down su istanze persona.

- Dataset: `train2017`, `val2017`, `person_keypoints_train2017.json`, `person_keypoints_val2017.json`.
- Training: usare le ground-truth bbox COCO per generare person crop.
- Test: per primo prototipo usare GT bbox per misurare solo la qualità del pose estimator; in seguito usare detector boxes per valutazione end-to-end.
- Input size consigliata: 256×192 per allinearsi ai checkpoint ViTPose base/simple.
- Target: 17 heatmap gaussiane, una per keypoint COCO, con visibility mask.
- Loss minima: MSE pesata dalla visibilità.

```
loss = ((pred_heatmaps - target_heatmaps) ** 2) * target_weights
loss = loss.mean()
```

7. Training schedule consigliato

1. **Sanity baseline:** far girare HF ViTPose originale su batch COCO crop e verificare output heatmaps.
2. **Warm-up TIPT:** congelare `backbone` e `head`; allenare solo shape stream + cross-attention per 2-5 epoch.
3. **Fine-tuning parziale:** sbloccare ultimi blocchi del backbone se accessibili, mantenendo LR più basso per i pesi pretrained.
4. **Training completo leggero:** ottimizzare tutto con due LR group: nuovo modulo LR alto, ViTPose LR basso.

Esempio param groups:

```
optimizer = AdamW([
    {"params": new_modules.parameters(), "lr": 1e-4},
    {"params": pretrained_modules.parameters(), "lr": 1e-5},
], weight_decay=0.05)
```

8. Baseline e ablation obbligatorie

Esperimento	Scopo
HF ViTPose baseline	riferimento pulito
TIPT con fusione semplice	verificare utilita' shape stream senza cross-attention
TIPT cross-attention	contributo principale
TIPT freeze vs unfreeze	capire se il guadagno viene dai nuovi moduli o dal fine-tuning
Sobel vs grayscale	capire se i bordi sono davvero utili

9. Aggiunte successive, non nella prima versione

Implementare solo dopo che la versione minimale funziona.

1. **Obfuscation training**: aggiungere blur/pixelation online o dataset COCO-BLUR/COCO-PIX.
2. **Invariance loss**: usare due viste della stessa crop, ad esempio blur e pixelation, e imporre $F_{shape}^{blur} \approx F_{shape}^{pix}$.
3. **Gate dinamico**: sostituire α scalare con gate dipendente da intensita' blur/pixelation o statistiche dell'immagine.
4. **Multi-level fusion**: aggiungere cross-attention anche a layer intermedi, non solo alla fine.
5. **Geometric bias**: inserire bias spaziale/anatomico nell'attenzione dello shape stream.

10. Criteri di successo

Il prototipo e' corretto se:

- produce heatmap $B \times 17 \times H_{out} \times W_{out}$ compatibili con ViTPose;
- carica correttamente i pesi HF ViTPose pretrained;
- mantiene la baseline HF come confronto riproducibile;
- esegue training su COCO2017 con loss custom;
- salva checkpoint e logga AP/OKS su val2017 o, inizialmente, metriche proxy su heatmap/PCK.

11. File suggeriti

```
project/
models/
  tipt_vitpose.py      # TiptVitPoseForPoseEstimation
  shape_modules.py    # SobelEdgeLayer, ShapePatchEmbedding, ShapeEncoder
  fusion.py           # ShapeGuidedCrossAttention
data/
  coco_keypoints.py   # COCO crops + target heatmaps
  transforms.py       # crop, resize, affine transform, heatmap generation
train.py
eval_coco.py
configs/tipt_vitpose_hf_coco.yaml
```

12. Riferimenti tecnici

- Hugging Face ViTPose docs: https://huggingface.co/docs/transformers/en/model_doc/vitpose
- HF implementation: https://github.com/huggingface/transformers/blob/main/src/transformers/models/vitpose/modeling_vitpose.py
- COCO keypoints: <https://cocodataset.org/#keypoints-2017>